

栈

从基本概念到算法应用

围绕栈的定义、基本操作、存储实现、典型应用、算法联系和练习题，系统讲清楚这一专题。

基本概念

操作与实现

典型应用

算法联系

本次主要内容

- 1 栈的定义与核心特征
- 2 顺序栈、链栈与 **push / pop**
- 3 括号匹配、表达式、浏览器历史
- 4 **DFS**、递归调用栈、单调栈练习题

课程结构

围绕栈从概念、实现、应用、算法联系一路推进

01

认识栈

定义、特征、术语、生活类比

02

栈的实现

顺序栈、链栈、push、pop、代码

03

典型应用

括号匹配、表达式求值、浏览器历史

04

算法联系

DFS、递归调用栈、单调栈

05

能力训练

易错点、队列对比、专项练习、答案提示

SECTION 01

认识栈

先建立后进先出的直觉，再记术语和结构。

本节重点

- 1 定义
- 2 LIFO
- 3 术语
- 4 生活类比

什么是栈

先看动图，再记定义，理解会更稳

核心内容

栈是一种只允许在同一端插入和删除的线性表。

最后进入栈的元素最先离开。

这个规则通常记成 **LIFO**。

栈顶是唯一操作端。

LIFO 图

栈的后进先出过程

元素不断压入栈顶

A TOP

栈的后进先出过程

元素不断压入栈顶

B TOP
A

栈的后进先出过程

最新元素位于最上方

D TOP
C
B
A

栈的后进先出过程

最后进入的 D 先离开

C TOP
B
A

栈顶

负责 push 和 pop

栈底

通常保持相对固定

栈的核心特征

抓住规则之后，术语和实现都会自然顺下来

核心内容

所有操作围绕栈顶进行。

最后进入的元素最先出来。

受限线性表的重点是“受限”两个字。

很多题的本质是识别处理顺序。

一句话

判断是不是栈，先看谁后进先出

后进先出

栈的后进先出过程

元素不断压入栈顶

A TOP

栈的后进先出过程

最新元素位于最上方

D TOP
C
B
A

栈的后进先出过程

元素不断压入栈顶

B TOP
A

栈的后进先出过程

最后进入的 D 先离开

C TOP
B
A

迁移

撤销、回退、递归都与它相关

栈的基本术语

术语不是背诵，而是帮助你精确描述元素位置

核心内容

栈顶：允许插入和删除的一端。

栈底：与栈顶相对的一端。

push：把元素压入栈顶。

pop：弹出并返回当前栈顶元素。

术语和位置对应

栈的后进先出过程

元素不断压入栈顶

A TOP

栈的后进先出过程

元素不断压入栈顶

B TOP
A

栈的后进先出过程

最新元素位于最上方

D TOP
C
B
A

栈的后进先出过程

最后进入的 D 先离开

C TOP
B
A

高频误区

读取栈顶不等于删除栈顶

表达建议

讲过程时始终围绕栈顶描述

生活中的栈

先用生活类比建立直觉，再转回程序结构

核心内容

托盘和书本都体现后进先出。

编辑器撤销依赖最近操作先被取消。

浏览器后退依赖最近页面状态先恢复。

生活类比的作用是帮助理解，不是替代定义。

状态回退类比

浏览器前进后退的双栈模型

当前页面：首页

后退栈

前进栈

TOP

浏览器前进后退的双栈模型

当前页面：课程页

后退栈

前进栈

首页

TOP

浏览器前进后退的双栈模型

当前页面：课程页

后退栈

前进栈

首页

TOP

练习页

浏览器前进后退的双栈模型

当前页面：练习页

后退栈

前进栈

课程页

TOP

首页

类比价值

帮助抽象概念落地

迁移目标

把现象转回程序中的状态管理

SECTION 02

栈的实现

把抽象规则落到数组、链表和代码上。

本节重点

- 1 顺序栈
- 2 链栈
- 3 push
- 4 pop

顺序栈

顺序栈最适合入门，因为数组和 **top** 指针最直观

核心内容

底层通常使用数组实现。

需要 **top** 指针指向当前栈顶。

容量固定时可能上溢。

访问速度快，实现简单。

优点

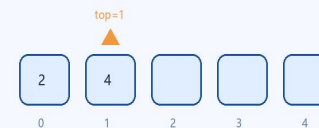
实现简单、局部性好

top 指针

顺序栈中 top 指针的移动



顺序栈中 top 指针的移动



顺序栈中 top 指针的移动



顺序栈中 top 指针的移动



不足

容量常常需要提前设定

入栈 push

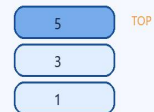
顺序要固定：判满、移动 top、写入元素

push(x)

```
def push(data, top, x, max_size):  
    if top == max_size - 1:  
        return top, False  
    top = top + 1  
    data[top] = x  
    return top, True
```

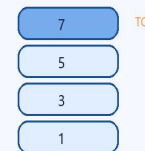
push

push(7) 动图



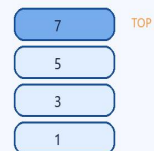
栈未满，准备压入 7
顺序：判满 -> top+1 -> 写入元素

push(7) 动图



top 先上移，新元素写入
顺序：判满 -> top+1 -> 写入元素

push(7) 动图



7 成为新的栈顶
顺序：判满 -> top+1 -> 写入元素

关键不是背代码，而是记住执行顺序不能写反。

出栈 pop

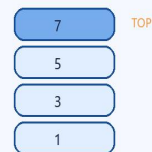
最容易错的是先取值还是先移动 top

pop()

```
def pop(data, top):  
    if top == -1:  
        return top, None  
    value = data[top]  
    top = top - 1  
    return top, value
```

pop

pop() 动图



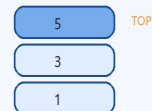
当前栈顶元素是 7
顺序: 判空 -> 读取栈顶 -> top-1

pop() 动图



先读取 7, 再准备移动 top
顺序: 判空 -> 读取栈顶 -> top-1

pop() 动图



top 下移, 5 成为新栈顶
顺序: 判空 -> 读取栈顶 -> top-1

pop 的重点是先读当前栈顶, 再让 top 下移。

链栈

链栈用链表头部充当栈顶，适合动态扩展

核心内容

每个结点通常包含 **data** 和 **next**。

入栈和出栈都在链表头部进行。

元素数量变化大时更灵活。

代价是每个结点都需要指针域。

适合场景

元素数量不固定

链栈

链栈按需扩展的过程



结点数量可以动态变化
适合元素个数变化较大的场景

链栈按需扩展的过程



结点数量可以动态变化
适合元素个数变化较大的场景

链栈按需扩展的过程



结点数量可以动态变化
适合元素个数变化较大的场景

代价

结点有额外空间开销

顺序栈与链栈对比

两种实现都正确，只是空间组织方式不同

顺序栈

底层连续空间，访问快。

实现简单，适合入门。

容量通常要提前确定。

容易出现上溢问题。

链栈

结点按需申请，更灵活。

适合元素数量波动大的场景。

每个结点额外保存指针。

逻辑更灵活，但实现略复杂。

共同点

都遵循后进先出

选择原则

看容量是否固定、实现是否追求简单

SECTION 03

典型应用

讲清楚栈怎么在真实问题里发挥作用。

本节重点

- 1 括号匹配
- 2 表达式求值
- 3 浏览器历史
- 4 状态回退

括号匹配

这是最经典的栈应用，也是考试高频

核心内容

左括号出现时直接入栈。

右括号出现时和栈顶左括号配对。

类型不匹配或栈空都说明表达式非法。

扫描结束后栈空才说明完全匹配。

例子

$\{ \}$ 合法， $()$ 不合法

括号匹配

括号匹配扫描过程

当前字符：开始
表达式： $\{ \}$

左括号入栈，右括号和栈顶配对
扫描结束后栈空，说明匹配成功

TOP

括号匹配扫描过程

当前字符：}
表达式： $\{ \}$

左括号入栈，右括号和栈顶配对
扫描结束后栈空，说明匹配成功

[TOP
(

括号匹配扫描过程

当前字符：[
表达式： $\{ \}$

左括号入栈，右括号和栈顶配对
扫描结束后栈空，说明匹配成功

[TOP
(

括号匹配扫描过程

当前字符：)
表达式： $\{ \}$

左括号入栈，右括号和栈顶配对
扫描结束后栈空，说明匹配成功

TOP

本质

栈保存尚未配对的左括号

括号匹配代码

把扫描过程写成代码，才能真正用于做题

is_valid(expr)

```
pairs = {')': '(', ']': '[', '}': '{'}
stack = []
for ch in expr:
    if ch in '([{': stack.append(ch)
    elif not stack or stack.pop() != pairs[ch]:
        return False
return not stack
```

扫描过程

括号匹配扫描过程

当前字符: 开始
表达式: (([]))

左括号入栈，右括号和栈顶配对
扫描结束后栈空，说明匹配成功

TOP

括号匹配扫描过程

当前字符:)
表达式: (([]))

左括号入栈，右括号和栈顶配对
扫描结束后栈空，说明匹配成功

[TOP
(

括号匹配扫描过程

当前字符: [
表达式: ((([]))

左括号入栈，右括号和栈顶配对
扫描结束后栈空，说明匹配成功

[TOP
(

括号匹配扫描过程

当前字符:)
表达式: ((([]))

左括号入栈，右括号和栈顶配对
扫描结束后栈空，说明匹配成功

TOP

这类题最关键的是把“未完成配对的信息”存在栈里。

表达式求值

运算顺序复杂时，栈可以负责管理优先级

核心内容

中缀表达式同时存在括号和优先级。

运算符可以先暂存到栈里。

优先级高的运算通常更早处理。

也可以用运算数栈和运算符栈配合求值。

思维核心

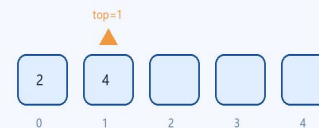
栈帮助我们处理“谁先算”

操作顺序示意

顺序栈中 top 指针的移动



顺序栈中 top 指针的移动



顺序栈中 top 指针的移动



顺序栈中 top 指针的移动



应用

编译器和计算器都离不开它

后缀表达式求值代码

讲表达式时要给一页可运行的思路代码

eval_postfix(tokens)

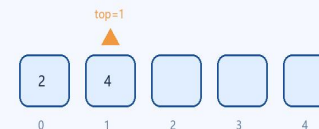
```
stack = []  
for token in tokens:  
    if token.isdigit():  
        stack.append(int(token))  
    else:  
        b = stack.pop(); a = stack.pop()  
  
    stack.append(eval(f'{a}{token}{b}'))  
return stack[-1]
```

顺序变化示意

顺序栈中 top 指针的移动



顺序栈中 top 指针的移动



顺序栈中 top 指针的移动



顺序栈中 top 指针的移动



操作数先存，运算符触发弹栈计算。

浏览器前进与后退

一个很生活化的例子，最适合讲状态回退

核心内容

当前页面后退时，要保存到前进栈中。

后退栈负责保存更早的访问状态。

点击前进时，从前进栈恢复最近状态。

两个栈配合，就能管理页面历史。

现实产品

浏览器、IDE、图形编辑器都很常见

双栈管理页面历史

浏览器前进后退的双栈模型



浏览器前进后退的双栈模型



浏览器前进后退的双栈模型



浏览器前进后退的双栈模型



SECTION 04

算法联系

把栈和图搜索、递归、单调优化真正连起来。

本节重点

- 1 DFS
- 2 递归栈
- 3 单调栈
- 4 代码模板

DFS 与显式栈

DFS 的本质是沿着一条路径不断深入，不通再回退

dfs(graph, start)

```
stack = [start]
```

```
visited = set()
```

```
while stack:
```

```
    node = stack.pop()
```

```
    if node in visited: continue
```

```
    visited.add(node)
```

```
    for nxt in reversed(graph[node]):
```

```
        stack.append(nxt)
```

DFS 栈变化

DFS 中显式栈的变化

访问 A, 压入起点

能继续深入就压栈, 走不通就弹栈回退

A TOP

DFS 中显式栈的变化

D 无路可走, 开始回退

能继续深入就压栈, 走不通就弹栈回退

B TOP

A

DFS 中显式栈的变化

继续深入到 B

能继续深入就压栈, 走不通就弹栈回退

B TOP

A

DFS 中显式栈的变化

回到 A 后转向 C

能继续深入就压栈, 走不通就弹栈回退

C TOP

A

显式栈版 DFS 能把“回退”过程看得非常清楚。

递归调用栈

递归函数看起来没写栈，但系统已经帮你维护了调用栈

fact(n)

```
def fact(n):  
    if n <= 1:  
        return 1  
    return n * fact(n - 1)
```

每次调用都会把现场压入调用栈

递归栈

递归调用栈的压入与弹出

调用开始
每次递归都把当前现场压入系统调用栈

fact(4) TOP

递归调用栈的压入与弹出

返回上一层
每次递归都把当前现场压入系统调用栈

fact(3) TOP

fact(4)

递归调用栈的压入与弹出

进入下一层
每次递归都把当前现场压入系统调用栈

fact(3) TOP

fact(4)

递归调用栈的压入与弹出

逐层回收现场
每次递归都把当前现场压入系统调用栈

fact(4) TOP

fact(4)

讲递归时，一定要明确：每次调用都在保存现场，返回时再恢复现场。

SECTION 05

能力训练

最后用对比、易错点和练习，确认是否真正掌握。

本节重点

- 1 易错点
- 2 栈队对比
- 3 练习题
- 4 答案提示

复杂度与边界条件

会写代码之前，先把边界和复杂度讲清楚

时间复杂度

Push: $O(1)$

Pop: $O(1)$

GetTop: $O(1)$

IsEmpty: $O(1)$

最常见错误

入栈前忘记判满。

出栈前忘记判空。

把 **top** 更新顺序写反。

把读取栈顶和删除栈顶混在一起。

一句话

栈之所以高效，是因为操作始终集中在栈顶

写题建议

先写边界判断，再写主体步骤

栈与队列的对比

核心差异不在名字，而在元素处理顺序

栈 Stack

后进先出 LIFO

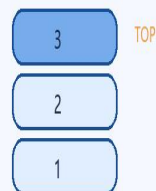
通常只在一端操作

适合回退、递归、括号匹配、单调栈

顺序差异

栈与队列的顺序差异

栈与队列的顺序差异



栈：后进先出

从左进入，从右离开



队列：先进先出

队列 Queue

先进先出，一端入队、一端出队

选择方法

先问谁应该先被处理，再决定用栈还是队列

练习一：括号匹配

练习题

- 1 判断表达式 $[(\{})]$ 是否匹配，并说明理由。
- 2 判断表达式 $([])$ 是否匹配，并指出在哪一步出错。
- 3 为什么扫描结束后还要检查栈是否为空？

提示

提示一：左括号先入栈。

提示二：右括号只和当前栈顶配对。

提示三：栈中残留左括号说明仍有未完成配对。

练习一参考答案

参考答案

([{}]) 匹配成功，因为每个右括号都能和当前栈顶的同类型左括号完成配对，最终栈为空。

([] 不匹配，因为扫描到) 时，当前栈顶是 [，类型不一致，立即判错。

扫描结束后检查栈是否为空，是为了确认没有残留未匹配的左括号。

练习二：出栈序列判断

练习页单独拆开，便于课堂停顿和提问

练习题

- 1 入栈序列为 **1,2,3,4,5**，序列 **4,5,3,2,1** 能否成为出栈序列？
- 2 入栈序列为 **1,2,3,4,5**，序列 **4,3,5,1,2** 能否成为出栈序列？
- 3 这类题为什么适合用模拟栈来判断？

提示

提示一：按入栈序列逐个压栈。

提示二：只要栈顶等于目标输出，就立刻出栈。

提示三：最后若目标序列无法完全匹配，就不是合法出栈序列。

练习二参考答案

答案不是只报结果，而是顺便回顾解题理由

参考答案

4,5,3,2,1 可以，因为先压入 1,2,3,4，弹出 4；再压入 5，弹出 5,3,2,1。

4,3,5,1,2 不可以，因为当 1 被要求输出时，2 还压在它上面，违背后进先出。

这类题适合模拟栈，是因为问题本身就在问“是否存在一组 push / pop 操作顺序”使目标序列成立。

谢谢大家!!!

真正学会栈，不是背定义，而是看到“回退、恢复、后进先出”就能主动想到它。